CAHIER DES CHARGES

Stage de Projet de Fin d'Année (PFA) — 2 mois

Assistant RAG multimodal avec serveur MCP pour l'interrogation de documents techniques

Intitulé du stage	Assistant RAG multimodal exposé via MCP — interrogation de documents techniques (texte + images)
Type de stage	Projet de Fin d'Année (PFA)
Durée	2 mois (8 semaines)
Stagiaire	Amine Amllal — Élève ingénieur IA & Technologies des Données, ENSAM Meknès
Domaine	Intelligence Artificielle — LLM, RAG, Vision par ordinateur, MCP (Model Context Protocol)

1. Contexte et problématique

Les organisations accumulent de grands volumes de documents techniques hétérogènes (rapports, manuels, procédures, comptes rendus) dont l'information est dispersée entre du texte, des tableaux, des schémas et des images. Retrouver une information précise dans ces corpus est chronophage et la recherche par mots-clés classique reste limitée.

Les approches RAG (Retrieval-Augmented Generation) permettent d'interroger ces documents en langage naturel avec des réponses sourcées. Toutefois, la plupart des pipelines RAG ignorent le contenu visuel (schémas, photos, captures), alors que celui-ci porte souvent une information clé. Le projet vise donc à concevoir un assistant RAG multimodal capable d'exploiter à la fois le texte et les images des documents.

Par ailleurs, afin de rendre le système interopérable et réutilisable, ses capacités seront exposées via le protocole MCP (Model Context Protocol). Le RAG devient ainsi un ensemble d'outils standardisés, directement consommables par n'importe quel client compatible MCP (assistant IA, application tierce), sans intégration spécifique.

2. Objectifs du stage

2.1 Objectif principal

Concevoir et développer un prototype fonctionnel d'assistant RAG multimodal permettant d'interroger un corpus de documents techniques et de restituer des réponses sourcées s'appuyant sur le texte et les images, et exposer ses capacités via un serveur MCP.

2.2 Objectifs spécifiques

- Mettre en place un pipeline d'ingestion de documents (PDF, Word) avec découpage sémantique (chunking) et indexation vectorielle.
- Extraire les images des documents et appliquer un OCR sur celles contenant du texte (Tesseract ou Azure Document Intelligence).

- Générer une description textuelle des images via un modèle de vision (VLM) afin de les rendre interrogeables.
- Indexer de manière unifiée les chunks de texte, le texte OCR et les descriptions d'images dans une même base vectorielle.
- Implémenter le module de retrieval et de génération avec citation des sources et affichage de l'image pertinente.
- Exposer les capacités du système via un serveur MCP fournissant des outils (recherche documentaire, récupération d'image source) consommables par un client compatible.
- Exposer le système également via une API REST et une interface de démonstration.
- Évaluer la qualité des réponses sur un jeu de test représentatif.

3. Périmètre fonctionnel

3.1 Fonctionnalités attendues

- Import et traitement automatique d'un lot de documents techniques.
- Recherche en langage naturel sur l'ensemble du corpus.
- Réponses générées accompagnées de leurs sources (extraits et images).
- Restitution des images sources pertinentes à côté de la réponse.
- Accès aux capacités du système via un serveur MCP (outils interrogeables depuis un client compatible tel que Claude Desktop).
- Tableau de bord simple de suivi de la qualité des réponses.

3.2 Hors périmètre

Compte tenu de la durée de 2 mois, les éléments suivants sont explicitement exclus : architecture multi-agent complexe, mécanisme d'auto-correction avancé (self/corrective RAG), authentification et gestion fine des droits, mise en production à grande échelle. Ces axes pourront constituer des perspectives d'extension.

4. Spécifications techniques

Brique	Technologies envisagées
Langage	Python
Orchestration RAG	LangChain
OCR	Tesseract ou Azure Document Intelligence
Vision (VLM)	GPT-4o vision, LLaVA ou BLIP
Base vectorielle	Chroma ou Qdrant
LLM de génération	OpenAI (GPT-4o) ou modèle open-source
Interopérabilité	Serveur MCP (Model Context Protocol) — SDK Python MCP
API	FastAPI
Interface (front-end)	React (+ Tailwind CSS)
Évaluation	RAGAS ou métriques personnalisées (faithfulness, pertinence)

5. Planning prévisionnel (8 semaines)

Semaines	Phase	Livrable intermédiaire
S1	Cadrage, état de l'art RAG multimodal, prise en main de la stack	Note de cadrage
S2	Ingestion texte + indexation vectorielle (RAG texte de base)	Pipeline texte fonctionnel
S3	Extraction des images + OCR	Module OCR opérationnel
S4	Description d'images via VLM + indexation unifiée	Index multimodal
S5	Retrieval multimodal + génération avec citations	Moteur de réponse
S6	Serveur MCP (outils RAG) + API FastAPI + interface React	Démo + serveur MCP
S7	Évaluation qualité + corrections	Rapport d'évaluation
S8	Finitions, documentation, préparation soutenance	Livrables finaux

6. Livrables attendus

- Code source documenté du pipeline d'ingestion multimodal (texte + OCR + images).
- Application de démonstration (serveur MCP + API FastAPI + interface web React).
- Jeu de test et rapport d'évaluation de la qualité des réponses.
- Documentation technique (architecture, installation, utilisation).
- Rapport de stage et support de soutenance.